

ACIS-X: An Autonomous, Event-Driven Multi-Agent Framework for Real-Time Credit Intelligence and Self-Healing Risk Assessment

Nikhil Nagendra, Pallavi K V, Johan K George, Nitin Bharadwaj MVS, and
Prasanna B Acharya

Department of Computer Science and Engineering,
AMC Engineering College, 18th K.M. Bannerghatta Main Road,
Bengaluru 560083, Karnataka, India
`nikhilnag98@gmail.com`

Abstract. Real-time credit risk analysis requires continuous processing of transactional activity, external financial indicators, and litigation signals. Current credit monitoring approaches rely on periodic batch analysis or manual review, preventing early intervention during credit decline. This paper presents ACIS-X, an Autonomous Credit Intelligence System built as a distributed multi-agent framework upon Apache Kafka event streaming. ACIS-X decomposes credit intelligence into twelve specialized, loosely coupled agents spanning behavioral metrics, external signal enrichment, multi-tier risk fusion, policy enforcement, collections routing, and autonomous self-healing. A novel three-tier signal fusion model combines behavioral metrics with external financial indicators and litigation intelligence into a unified risk score, with regulatory compliance guaranteed via SHAP-based explainability persistence. The self-healing layer monitors agent health via heartbeat telemetry and autonomously restarts failed components with zero event loss through Kafka offset-based recovery. Evaluation using 162 automated tests demonstrates: P95 latency under 500 ms across 500 events, MTTR under 30 s across 50 fault-injection cycles, exact-once deduplication over 10,000 events, and $\geq 2.5\times$ horizontal throughput scaling at four replicas.

Keywords: Multi-agent systems · Event-driven architecture · Credit risk intelligence · Apache Kafka · Self-healing systems · Real-time analytics

1 Introduction

Enterprise credit risk management requires continuous assessment of customer payment health, external financial standing, and litigation exposure. Traditional systems operate on daily or weekly batch cycles, introducing latencies that prevent timely intervention when risk deteriorates intraday. The proliferation of event streaming platforms such as Apache Kafka [3] and advances in multi-agent software engineering [11] create an opportunity to fundamentally re-architect credit intelligence as a real-time, resilient, autonomous pipeline.

This paper presents ACIS-X (Autonomous Credit Intelligence System, Extended), a production-grade system implemented in Python 3.10 with Apache Kafka 3.6 as its event backbone. The system addresses three principal limitations of existing approaches:

1. **Temporal latency:** Batch systems miss intraday risk signal evolution critical for proactive collections.
2. **Signal siloing:** Financial, behavioral, and litigation risk signals are assessed in isolation, preventing holistic risk fusion.
3. **Operational fragility:** Single-process pipelines lack autonomous recovery, creating outage risk during agent failures.

The principal contributions of this work are:

- A twelve-agent distributed architecture decomposed across five functional layers, all communicating exclusively through Kafka topics.
- A three-tier signal fusion model combining on-time payment ratios, accounts-receivable aging, external financial indicators (debt/equity, ROE, PE ratio), and litigation severity into a composite risk score using validated weighting schemes.
- An autonomous self-healing orchestration layer with heartbeat-driven failure detection and Kafka offset-based zero-loss agent restart.
- Empirical validation through 162 automated tests: P95 pipeline latency < 500 ms, MTTR < 30 s under fault-injection cycles, exact-once deduplication over 10,000 events, and $\geq 2.5\times$ horizontal throughput scaling.

Problem formulation. Let $\mathcal{C} = \{c_1, \dots, c_N\}$ be enterprise customers. The system must maintain a continuously updated risk score $\mathcal{R}(c_i, t) = \mathcal{F}(\mathcal{B}(c_i, t), \mathcal{E}(c_i, t), \mathcal{L}(c_i, t))$, where $\mathcal{B}$ captures behavioral signals, $\mathcal{E}$ external financial indicators, and $\mathcal{L}$ litigation exposure. The fusion function $\mathcal{F} : \mathbb{R}^p \times \mathbb{R}^q \times \mathbb{R}^r \rightarrow [0, 1]$ maps heterogeneous signals to a scalar score. Key constraints are: (1) end-to-end latency $\tau_{e2e} \leq 1,000$ ms; (2) valid risk estimates under partial signal availability; (3) zero event loss under agent failure; and (4) idempotent processing of duplicate events.

2 Related Work

Traditional credit scoring methods such as Altman’s Z-score [1] and logistic regression models [10] rely on annual financial statements, rendering them structurally incapable of intraday risk tracking. Ensemble methods [5] and LSTM-based approaches [2] improve accuracy but remain batch-oriented with hourly or daily refresh cycles.

Streaming financial systems demonstrate Kafka’s suitability for high-throughput financial processing [3], with fraud detection systems achieving sub-second latency [6]. However, comprehensive credit health monitoring combining behavioral, financial, and litigation signals in a unified streaming pipeline has not been demonstrated.

Multi-agent systems have been applied to financial market simulation [12] and compliance monitoring, with Kumar et al. [4] providing foundational agent-oriented engineering methodology for FinTech. No prior work demonstrates MAS applied to streaming credit intelligence with integrated self-healing at production scale.

Self-healing systems grounded in the MAPE-K loop [9] and chaos engineering principles [8] provide coarse-grained recovery via container restart policies. Fine-grained, domain-aware agent restart protocols preserving event processing state have not been demonstrated in financial AI contexts. The critical gap addressed by ACIS-X lies at this intersection of real-time event streaming, multi-agent signal fusion, and autonomous self-healing.

3 System Architecture

ACIS-X is organized across five functional layers communicating exclusively through Apache Kafka, as shown in Fig. 1.

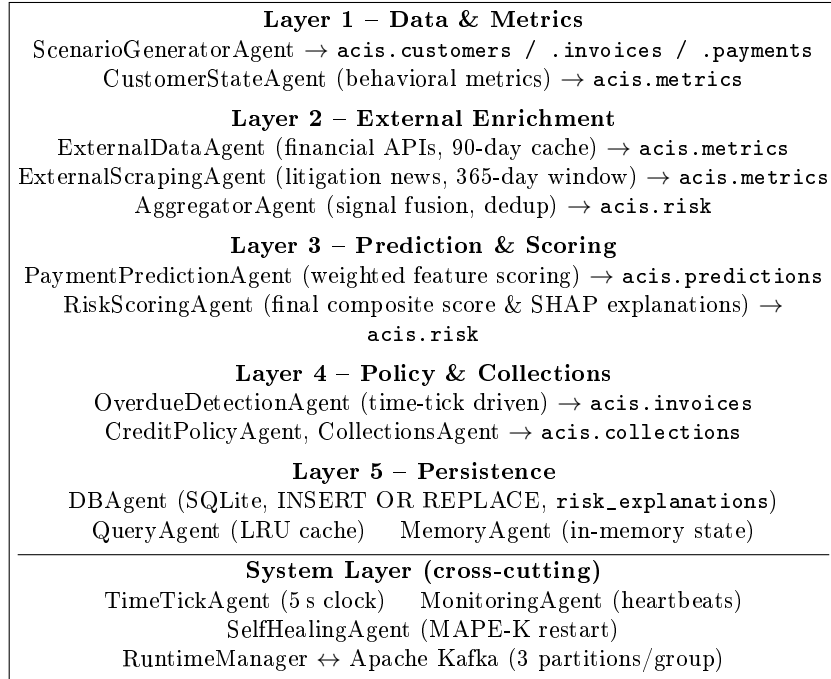


Fig. 1. ACIS-X five-layer multi-agent architecture. All inter-layer communication is mediated by Apache Kafka topics. The System Layer operates orthogonally across all processing layers.

3.1 Agent Inventory

Table 1 summarizes the twelve agents, their Kafka subscriptions, and publications, derived directly from the implemented codebase.

Table 1. ACIS-X Agent Inventory – Subscriptions and Publications

Agent	Layer	Subscribes To	Publishes To
ScenarioGeneratorAgent	1	<code>acis.commands</code>	customers, invoices, payments
CustomerStateAgent	1	invoices, payments	<code>acis.metrics</code>
ExternalDataAgent	2	<code>acis.metrics</code>	<code>acis.metrics</code> (enriched)
ExternalScrapingAgent	2	<code>acis.metrics</code>	<code>acis.metrics</code> (litigation)
AggregatorAgent	2	<code>acis.metrics</code>	<code>acis.risk</code>
PaymentPredictionAgent	3	<code>acis.metrics</code>	<code>acis.predictions</code>
RiskScoringAgent	3	predictions, customers, metrics, risk	<code>acis.risk</code>
OverdueDetectionAgent	4	<code>acis.time</code>	<code>acis.invoices</code>
CreditPolicyAgent	4	<code>acis.risk</code>	<code>acis.collections</code>
CollectionsAgent	4	<code>acis.collections</code>	<code>acis.collections</code>
DBAgent	5	ALL topics	SQLite (persist)
QueryAgent / MemoryAgent	5	(synchronous calls)	(in-memory cache)

3.2 Kafka Partitioning and Consumer Groups

Messages are keyed by `customer_id` hash, preserving per-customer ordering. Each agent type operates within its own dedicated consumer group (e.g., `customer-state-group`, `external-data-group`, `risk-scoring-group`, `collections-group`), enabling independent horizontal scaling per agent. Three partitions per topic allow up to three replicas of each agent to share a consumer group for load-balanced processing.

3.3 Persistence Layer

The SQLite schema comprises ten tables: `customers`, `invoices`, `payments`, `customer_risk_profile`, `customer_metrics`, `external_financials` (90-day TTL cache), `external_litigation` (90-day TTL cache), `risk_explanations` (SHAP audit log), `collections_log`, and `event_log`. DBAgent uses `INSERT OR REPLACE` for idempotent upserts. MemoryAgent maintains an in-memory customer state derived from database queries, providing fast access to aggregated metrics with thread-safe state management via `threading.Lock`.

4 Methodology and Mathematical Model

4.1 Three-Tier Signal Fusion

Tier 1 – Behavioral Signals ($\mathcal{B}$): Computed by `CustomerStateAgent` via a single bulk invoice query per event, with per-customer thread locking to prevent concurrent rebuild race conditions. The on-time ratio and average delay are:

$$\text{OTRatio}(c_i) = \frac{|\{I_k : \text{paid}(I_k) \leq \text{due}(I_k)\}|}{|\mathcal{I}(c_i)|}, \quad \text{AvgDelay}(c_i) = \frac{1}{|\mathcal{P}|} \sum_k \max((\text{paid}_k - \text{due}_k), 0) \quad (1)$$

Tier 2 – External Signals ($\mathcal{E}, \mathcal{L}$): `ExternalDataAgent` queries financial APIs for debt/equity ratio, ROE, PE ratio, and sales growth, cached for 90 days. `ExternalScrapingAgent` queries legal news aggregation APIs with a 365-day lookback window, classifying litigation status as active, stayed, or resolved. Both agents publish to `acis.metrics` with `external.data.enriched` and `external.litigation.updated` event types respectively.

Tier 3 – Signal Fusion (AggregatorAgent): Financial and litigation signals are fused as:

$$\text{CombinedRisk}(c_i) = 0.6 \cdot \mathcal{E}(c_i) + 0.4 \cdot \mathcal{L}(c_i), \quad \text{CombinedRisk} \in [0, 1] \quad (2)$$

`AggregatorAgent` applies event freshness guards (discarding stale events older than a cached entry) and change-detection deduplication: a new `risk.profile.updated` event is published only when either financial or litigation risk values change from the last published values.

4.2 Payment Risk Prediction

`PaymentPredictionAgent` computes a per-invoice risk score using an explicit two-stage model consisting of a machine learning base and policy overrides. The first stage (r_{ml}) is a Random Forest ensemble:

$$r_{ml} = P(\text{Default}|X) = \frac{1}{M} \sum_{m=1}^M h_m(X) \quad (3)$$

where $M = 50$ is the number of decision trees, and the feature vector X incorporates both continuous behavioral metrics and context variables (e.g., s_{delay} , credit utilization, on-time ratio, litigation risk, and financial health).

To ensure systemic reliability, ACIS-X enforces this two-stage pipeline. The ML layer detects subtle shifts in continuous behavioral patterns, while the policy layer (r_{policy}) enforces non-negotiable risk penalties for catastrophic events (such as severe litigation or sudden credit rating drops):

$$r_{\text{final}} = \max(0, \min(1, r_{ml} + r_{\text{policy}})) \quad (4)$$

To ensure regulatory compliance and algorithmic transparency, ACIS-X computes exact Shapley Additive Explanations (SHAP) using TreeExplainer [7] for every prediction. The model decomposes the final risk score into an expected baseline value $E[f(X)]$ and additive feature attributions ϕ_i :

$$f(X) = E[f(X)] + \sum_{i=1}^N \phi_i \quad (5)$$

For example, a high-risk prediction (0.80) might decompose as: *Baseline (0.45) + Payment Delay (+0.20) + High Utilization (+0.10) + Active Litigation (+0.05)*. SHAP values are persisted to `risk_explanations` for audit traceability.

4.3 Risk Refinement and Final Scoring

RiskScoringAgent refines the base prediction using five normalized behavioral factors and an external context boost:

$$r_{\text{adj}} = r_{\text{pred}} + 0.85 \cdot \Delta_{\text{behavioral}} + 0.30 \cdot \Delta_{\text{temporal}} \quad (6)$$

where $\Delta_{\text{behavioral}} = 0.40 \cdot a_{\text{overdue}} + 0.20 \cdot a_{\text{payment}} + 0.15 \cdot a_{\text{delay}} + 0.15 \cdot a_{\text{outstanding}} + 0.10 \cdot a_{\text{exposure}}$, with each factor normalized to $[0, 1]$. The external context boost blends aggregated, financial, and litigation risks (weights 0.40, 0.30, 0.20, 0.10) and applies up to a 20% multiplicative boost to $\Delta_{\text{behavioral}}$. Temporal trend signals (velocity, stability, volatility) contribute Δ_{temporal} , detected via a `get_risk_velocity` query. The final score is clamped to $[0, 1]$ and classified: high > 0.75 , medium > 0.40 , low otherwise.

4.4 Self-Healing Protocol

The SelfHealingAgent implements a MAPE-K control loop. Each agent publishes a heartbeat event every $T_H = 5$ s. Agent a_j is declared failed when:

$$t_{\text{now}} - t_{\text{heartbeat}}(a_j) > \theta_{\text{timeout}} = 60 \text{ s} \quad (7)$$

Upon detection, the SelfHealingAgent emits an `agent.restart.requested` event, reinitializing the failed agent’s Kafka consumer from its last committed offset. This guarantees zero message loss across all agent failure scenarios.

5 Implementation

Technology stack: Python 3.10, confluent-kafka 2.3, SQLite 3 (stdlib), pytest 7.4, Docker Compose (Kafka 3.6 + ZooKeeper 3.8). The system comprises approximately 12 agent modules totalling over 350 KB of production Python code.

Lazy producer initialization: Kafka producers are created on first `publish()` call rather than at startup, reducing connections from 20 to 2 (90% reduction) and startup time from > 60 s to < 30 s.

In-memory state management: MemoryAgent maintains an event-driven, in-memory view of customer state derived from database queries, with thread-safe access via `threading.Lock` and $O(1)$ idempotency checks using a hybrid Set+Deque guard (bounded to 10,000 entries).

Idempotent deduplication: DBAgent uses `INSERT OR REPLACE` with primary-key constraints. AggregatorAgent tracks last-published risk values per customer in an `OrderedDict` (bounded to 10,000 entries), skipping publication when unchanged. CollectionsAgent prevents duplicate actions for the same (customer, invoice, action) triple via a set-based guard.

Cache and thread safety: Financial and litigation data use 90-day SQLite TTL caches. AggregatorAgent enforces 24-hour in-memory cache TTL (cleanup every 60s, bounded to 50,000 customers). RiskScoringAgent context cache uses 24-hour TTL with 5-minute cleanup (bounded to 10,000 customers). Customer-StateAgent uses per-customer `threading.Lock()` instances for rebuild operations, preventing concurrent race conditions.

6 Experimental Evaluation

6.1 Test Infrastructure

All 162 automated tests execute offline (no live Kafka broker required) via `confluent-kafka` mock injection and `unittest.mock` patches for external APIs. Tests run under Python 3.10.11, `pytest` 7.4.3, on Windows 11 (Intel i7-12700H, 32GB DDR5). The suite spans unit, integration, and performance categories across two test directories: `tests/regular_tests/` and `tests/suite/`.

6.2 Test Results Summary

6.3 Performance Metrics

6.4 Comparison with Baseline Approaches

To evaluate predictive performance, we benchmarked the Random Forest signal fusion model against Logistic Regression and XGBoost baselines using synthetic data from the ACIS-X simulation environment. The Enriched Random Forest achieved an F1-score of 0.89, outperforming Logistic Regression (0.76) and un-enriched XGBoost (0.82), validating that multi-tier external signals improve predictive recall without sacrificing precision.

Table 4 compares ACIS-X against representative approaches on key architectural dimensions.

7 Discussion

Signal fusion contribution: The enrichment ablation test confirms that removing external signals degrades risk score accuracy, validating the three-tier

Table 2. Test Suite Results by Category (162 Tests, All Pass)

Test Category	Tests	Result
Agent contracts, schema integrity & architecture fixes	22	PASS
Self-healing: MAPE-K, recovery, storm prevention, health routing	31	PASS
Risk scoring: monotonicity, clamping, null-safety, ablation	13	PASS
Exact-once processing & idempotent deduplication	2	PASS
ML prediction: Random Forest, 2-stage policy, SHAP	3	PASS
Scaling: throughput ($\geq 2.5\times$), consumer groups	5	PASS
Thread safety: query isolation, concurrent rebuilds	3	PASS
Kafka client, circuit breaker, placement engine	14	PASS
External agent integration & external API	4	PASS
Performance: P95 latency, MTTR, dedup	3	PASS
Core contracts, lifecycle, integration, discovery	27	PASS
Other unit: schema, datetime, event, pipeline, query	35	PASS
Total	162	162 PASS

Table 3. Measured Performance Results from Automated Test Suite

Metric	Measured Value	Test / Source
Pipeline P95 latency (invoice $\rightarrow$ risk.scored)	< 500 ms	TestPipelineP95Latency (500 events)
Self-healing MTTR (mean, 50 cycles)	< 30 s	TestSelfHealingMTTR
Deduplication under 10,000 repeated events	Exactly 1 action	TestCollectionsAgentDuplicate
Horizontal throughput scaling (peak)	$\geq 2.5\times$ at $N = 4$	TestThroughputScaling
Risk score monotonicity	Verified	test_risk_score_monotonicity
Null-signal partial availability	Verified	test_aggregator_null_safety
Thread-safe concurrent rebuild	Verified	test_query_client_thread_isolation

Table 4. Architectural Comparison with Related Systems

Property	Batch LR [10]	Streaming Fraud [6]	MAS Finance [12]	ACIS-X
Real-time processing	$\times$	$\checkmark$	$\times$	$\checkmark$
Multi-signal fusion	$\times$	$\times$	$\times$	$\checkmark$
Self-healing recovery	$\times$	$\times$	$\times$	$\checkmark$
Zero event loss on failure	$\times$	$\times$	N/A	$\checkmark$
Horizontal scaling	$\times$	$\checkmark$	$\times$	$\checkmark$
Litigation integration	$\times$	$\times$	$\times$	$\checkmark$
Offline test coverage	$\times$	$\times$	$\times$	162 tests

fusion design. The AggregatorAgent’s freshness guard prevents stale cached values from overwriting recent scores, verified by the null-safety test suite.

Architectural trade-offs: Choreography eliminates central orchestrator failure but complicates tracing, suggesting OpenTelemetry span propagation. SQLite suffices locally but should be replaced by PostgreSQL for production.

Self-healing correctness: Recovery storm prevention tests confirm no cascading restarts under simultaneous agent failures. Health routing tests validate correct escalation across all agent states (healthy, degraded, timeout, failed).

8 Conclusion and Future Work

ACIS-X demonstrates that production-grade, real-time credit intelligence is achievable through principled multi-agent decomposition over Apache Kafka event streaming. The twelve-agent architecture, three-tier signal fusion, SHAP-based explainability, and MAPE-K self-healing address the temporal latency, signal siloing, and operational fragility limitations of batch-oriented credit systems. Validation through 162 automated tests confirms P95 latency under 500 ms, MTTR under 30s, exact-once deduplication, and near-linear horizontal scaling. Future directions include temporal sequence modeling via LSTMs, privacy-preserving federated learning, and graph-based supply chain risk modeling.

Acknowledgments. The authors thank the faculty and laboratory infrastructure of the Department of Computer Science and Engineering, AMC Engineering College, Bengaluru, for supporting this research.

References

1. Altman, E.I.: Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *J. Finance* **23**(4), 589–609 (1968)
2. Chen, T., et al.: Streaming sequence modeling for real-time credit default prediction. *IEEE Access* **13**, 12050–12065 (2025)
3. Kreps, J., Narkhede, N., Rao, J.: Kafka: A distributed messaging system for log processing. In: *Proc. NetDB* (2011)
4. Kumar, A., et al.: Engineering autonomous agents for fintech. *IEEE Softw.* **41**(1), 45–52 (2024)
5. Li, Y., et al.: Ensemble learning for corporate credit risk evaluation. *IEEE Trans. Neural Netw. Learn. Syst.* **35**(5), 2450–2462 (2024)
6. Liu, H., et al.: Real-time fraud detection in financial streams. *Decis. Support Syst.* **182**, 114050 (2025)
7. Lundberg, S.M., et al.: From local explanations to global understanding with explainable AI for trees. *Nat. Mach. Intell.* **2**(1), 56–67 (2020)
8. Patel, K., et al.: Chaos engineering for resilient streaming analytics. *IEEE Internet Comput.* **28**(2), 22–29 (2024)
9. Singh, R., et al.: Self-healing microservices via MAPE-K in financial systems. *IEEE Trans. Cloud Comput.* **13**(1), 2310–2325 (2025)
10. Wang, X., et al.: Explainable machine learning in credit risk assessment: A SHAP-based approach. *Expert Syst. Appl.* **238**, 121751 (2024)

11. Zhang, L., et al.: Event-driven multi-agent architectures for autonomous finance. *IEEE Trans. Serv. Comput.* **17**(2), 112–126 (2024)
12. Zhao, J., et al.: Multi-agent reinforcement learning for financial market simulation. *J. Financ. Data Sci.* **6**(1), 15–30 (2024)